Java Interview Questions and Answers

1. What is the history of Java?

Answer:

- Java was originally developed by **James Gosling** and his team at **Sun Microsystems** in **1991** as part of the *Green Project*.
- Initially named **Oak** (after an oak tree outside Gosling's office).
- Renamed **Java** in **1995**, inspired by Java coffee.
- In **2009**, Sun Microsystems was acquired by **Oracle**, which now owns Java.
- It was designed with the principle **"Write Once, Run Anywhere" (WORA)**.

2. What are the main features of Java?

Answer:

- **Simple:** Easy to learn, similar to C/C++.
- **Object-Oriented:** Everything revolves around objects.
- **Platform Independent:** Compiles into **bytecode**, which runs on any JVM.
- **Secure:** No explicit pointers, provides security manager & bytecode verification.
- **Robust:** Strong memory management and exception handling.
- **Multithreaded:** Built-in support for concurrent execution.
- **High Performance:** JIT (Just-In-Time) compiler boosts performance.
- **Distributed:** Supports RMI, EJB for distributed applications.

3. What is the difference between JDK, JRE, and JVM?

Answer:

- **JDK (Java Development Kit):**
  - Complete development toolkit.
  - Contains JRE + development tools (compiler `javac`, debugger, etc.).
  - Used by developers to write, compile, and debug Java programs.
- **JRE (Java Runtime Environment):**
  - Provides runtime environment to execute Java applications.
  - Contains JVM + core libraries but no compilers/debuggers.

- Used by end users who just want to run Java apps.

- **JVM (Java Virtual Machine):**
  - Abstract machine that executes **bytecode**.
  - Provides platform independence by interpreting bytecode into native machine code.
  - Handles memory management (Garbage Collection).

4. What is PATH in Java?

Answer:

- **PATH** is an environment variable that tells the operating system **where to find executables**.
- When we type `javac` or `java` in the command line, PATH ensures the system can locate these programs from the JDK's `bin` directory.

5. What is CLASSPATH in Java?

Answer:

- **CLASSPATH** is an environment variable that tells JVM **where to find .class files or packages**.
- By default, JVM looks in the current directory.
- We set CLASSPATH when using external libraries or user-defined packages.

6. What is a `.class` file in Java?

Answer:

- When you compile a Java program (`.java` file), the **Java compiler (`javac`) generates a `.class` file**.
- This file contains **bytecode**, which is a platform-independent intermediate representation of the code.

7. What is Bytecode in Java?

Answer:

- **Bytecode** is the intermediate machine-independent code generated after compilation.
- It is stored in `.class` files.
- The **JVM interprets or JIT compiles bytecode** into machine code specific to the underlying operating system and CPU.
- This is what makes Java **platform independent**.

8. Why is Java called Platform Independent?

Answer:

- Java code is compiled into **bytecode**, not machine code.
- This bytecode can run on any machine that has a **JVM installed**, regardless of OS (Windows, Linux, macOS).
- Hence, the same Java program can run anywhere → **Write Once, Run Anywhere (WORA)**.

9. Explain the execution process of a Java program.

Answer:

1. **Write Code:** Save source code in `.java` file.
2. **Compile:** `javac MyProgram.java` → generates `MyProgram.class` (bytecode).
3. **Run:** `java MyProgram` → JVM loads `.class` file and executes bytecode.

10. What is the role of the JVM in security?

Answer:

- JVM verifies bytecode before execution (Bytecode Verifier).
- No explicit pointers → prevents unauthorized memory access.
- Security Manager allows applications to define rules (like file access, network access).

1. **JDK vs JRE vs JVM**

Aspect	JDK (Java Development Kit)	JRE (Java Runtime Environment)	JVM (Java Virtual Machine)
Definition	Toolkit for developing Java programs	Environment to run Java programs	Abstract machine to execute bytecode
Contains	JRE + Development tools (javac, debugger, etc.)	JVM + Core libraries	Execution engine only
Purpose	For developers (coding, compiling, debugging)	For end users (running apps)	For converting bytecode → machine code
Includes Compiler?	✅ Yes (`javac`)	❌ No	❌ No
Includes Libraries?	✅ Yes	✅ Yes	❌ No
Platform Independent?	Partially (depends on OS JDK)	Partially (depends on OS JRE)	✅ Yes (executes bytecode on any OS)

2. **PATH vs CLASSPATH**

Aspect	PATH	CLASSPATH
Definition	Tells OS where to find executables (javac, java, etc.)	Tells JVM where to find `.class` files and libraries
Used By	Operating System	Java Virtual Machine
Default	Must be set to JDK `bin` folder	Current directory (.)
Example	`C:\Program Files\Java\jdk-21\bin`	`.;C:\lib\mysql-connector.jar`
Error if not set?	`javac' is not recognized…`	`ClassNotFoundException`

3. **.java file vs .class file vs Bytecode**

Aspect	`.java` file	`.class` file	Bytecode
Generated By	Programmer (source code)	Compiler (`javac`)	Compiler inside `.class`
Content	Human-readable Java code	Bytecode instructions	JVM-understandable instructions
Readable By	Humans	JVM	JVM only
Example	`public class Test {}`	`Test.class`	`aload_0`, `invokevirtual`, etc.